

TUGAS GENERATIVE AI

Nama : Putri Torsia Aura Prameruary

NIM : 2546000083

Kelas : Pembelajaran Mesin - B

Halaman Ke-1

B. Ketentuan Teori dan Konsep

1.) Apa itu Generative AI?

Generative AI adalah model yang belajar dari data lalu menggunakan hasil belajarnya untuk menciptakan data baru yang belum pernah ada sebelumnya. Model ini tidak hanya mengenali pola, tetapi juga bisa mengimajinasikan output baru seperti gambar, teks, atau audio. Kuncinya ada pada kata distribusi: model mempelajari distribusi data, lalu mengambil sampel dari distribusi itu untuk menghasilkan sesuatu yang baru tetapi tetap masuk akal.

2.) Apa perbedaan discriminative model dan generative model?

Model discriminative belajar membedakan kelas yaitu menjawab "data ini masuk kategori mana?". Model ini tidak peduli bagaimana data terbentuk, hanya peduli pada keputusan antar kelas. Model generative belajar distribusi data itu sendiri yaitu menjawab "data seperti apa yang mungkin muncul dari distribusi ini?". Dari pemahaman itulah model bisa menciptakan data baru. Yang dapat dianalogikan sebagai berikut: discriminative seperti hakim yang memilah dokumen asli vs palsu, sedangkan generative seperti seniman yang bisa membuat dokumen baru yang terlihat asli.

3.) Apa perbedaan klasifikasi dan generasi?

- Klasifikasi: Input ke label. Contoh: gambar kucing masuk, keluar label "kucing".
- Generasi: noise atau latent vector atau prompt ke output baru. Contoh: vektor acak masuk, keluar gambar kucing baru.

Pada klasifikasi model memilih, sedangkan pada generasi model menciptakan.

4.) Mengapa model generative dapat membuat data baru?

Karena selama training, model mempelajari struktur statistik tersembunyi dari data seperti pola warna, bentuk, dan ukuran kata. Setelah distribusi itu dipahami, model bisa mengambil sampel baru darinya. Seperti seseorang yang sudah membaca ribuan kalimat lalu bisa menulis kalimat baru yang tidak pernah ia baca sebelumnya, tetapi tetap masuk akal secara gramatikal.

5.) Apa itu latent space?

Latent space adalah ruang representasi tersembunyi berukuran kecil yang mewakili inti dari data.

Misalnya gambar 128×128 pixel dikompresi encoder menjadi vektor z berukuran 128 dimensi saja. Vektor kecil ini merangkum fitur penting seperti bentuk, warna, dan gaya tanpa menyimpan semua detail pixel.

Yang menarik adalah dua gambar yang mirip akan punya vektor z yang berdekatan di latent space. Kita bisa berjalan antar dua titik di latent space dan hasilnya adalah transisi gambar yang mulus.

6.) Apa itu sampling?

Sampling adalah proses mengambil nilai secara acak dari distribusi probabilitas yang sudah dipelajari model.

Di VAE misalnya, sampling dilakukan dengan mengambil z dari $N(\mu, \sigma^2)$ lalu memasukkannya ke decoder. Di text generation, sampling berarti memilih token berikutnya dari distribusi probabilitas antar

Nama: Putri Torsta Aura Prameswary

Nim: 2546000083

Kelas: Pembelajaran mesin - B

Halaman ke-2

Vocabulary. Tanpa sampling, model hanya mereproduksi data lama. Dengan sampling, setiap kali dijalankan model bisa menghasilkan output yang berbeda meskipun input awalnya sama.

7.) Apa itu noise vector?

Noise Vector (z) adalah vektor acak yang biasanya dari distribusi normal standar $N(0, I)$ dan menjadi titik awal proses generasi. Di GAN, noise vector inilah yang diubah generator menjadi gambar. Berbeda seed berarti berbeda noise vector yang berarti berbeda pula output yang dihasilkan. Bisa dianggap sebagai benih kreativitas model: dari benih yang berbeda, tumbuh output yang berbeda pula.

8.) Apa itu Prompt?

Prompt adalah instruksi atau konteks yang diberikan ke model sebagai kondisi c untuk mengarahkan outputnya. Secara matematis, prompt menggeser model dari menghasilkan $p(x)$ menjadi $p(x|c)$ yaitu distribusi yang lebih terarah. Penting untuk dipahami bahwa prompt bukan perintah absolut. Ia hanya menggeser distribusi sampling, bukan mengunci satu output tertentu. Itulah mengapa prompt yang hampir sama bisa menghasilkan output yang sedikit berbeda.

9.) Apa perbedaan image generation dan text generation?

- > Image generation menghasilkan output berupa matriks piksel dan bekerja di ruang kontinu. Arsitektur yang umum digunakan adalah GAN, VAE, dan diffusion Model. Tantangan utamanya adalah menjaga koherensi spasial antar piksel.
- > Text generation menghasilkan output berupa urutan token diskret. Model bekerja secara autoregressive yaitu satu token per langkah dimana tiap token bergantung pada token sebelumnya. Arsitektur yang umum adalah Transformer atau LLM. Tantangan utamanya adalah menjaga koherensi semantik dan gramatikal sepanjang urutan.

10.) Mengapa loss kecil belum tentu berarti hasil generasi bagus?

Loss hanya mengukur selisih numerik antara prediksi dan target, bukan kualitas atau realisme output secara keseluruhan. Beberapa contoh nyata:

•> Di VAE, loss MSE bisa kecil tetapi gambar hasil generasi tampak blur karena model cenderung mengambil nilai rata-rata piksel.

•> Di GAN, loss generator bisa rendah tetapi terjadi mode collapse yaitu model hanya menghasilkan satu jenis output terus-menerus.

•> Di LLM, Perplexity bisa rendah tetapi model tetap menghasilkan teks yang halusnasi atau tidak faktual.

Kualitas generasi harus dievaluasi secara terpisah menggunakan metrik FID untuk gambar atau BLEU / ROUGE untuk teks, atau bisa juga melalui evaluasi manusia secara langsung.

Nama: Putri Torsta Aura Prameswary

Nim: 2546000083

Kelas: Pembelajaran Mesin - B

Halaman Ke-3

C. Rumus yang Wajib Dipahami

1) Konsep Dasar Generative Model

$p(x)$ untuk unconditional, $p(x|c)$ untuk conditional

Variabel	Arti
x	Data yang ingin dihasilkan seperti gambar atau teks
c	Kondisi berupa prompt, label kelas, atau gambar referensi
$p(x)$	Probabilitas kemunculan data x tanpa kondisi apapun
$p(x c)$	Probabilitas menghasilkan x diberikan kondisi c .

2) Autoencoder

$z = f_{\theta}(x)$ lalu $\hat{x} = g_{\varphi}(z)$ lalu $L = \|x - \hat{x}\|^2$

Variabel	Arti
x	Input asli
z	Latent vector yaitu representasi terkompresi dari x
$\hat{x}$	Hasil rekonstruksi dari decoder
f_{θ}	Encoder dengan parameter θ
g_{φ}	Decoder dengan parameter φ
L	Reconstruction loss dihitung dengan MSE

Tabel Fungsi PyTorch

Rumus	Kode PyTorch
$z = f_{\theta}(x)$	<code>self.encoder(x)</code>
$\hat{x} = g_{\varphi}(z)$	<code>self.decoder(z)</code>
$L = \ x - \hat{x}\ ^2$	<code>nn.MSELoss</code>

3) Variational Autoencoder (VAE)

$z = \mu + \sigma \cdot \epsilon$, dengan $\epsilon \sim \mathcal{N}(0, I)$

$L = L_{\text{rekon}} + L_{\text{KL}}$, dimana $L_{\text{KL}} = D_{\text{KL}}(q_{\varphi}(z|x) || p(z))$

Variabel	Arti
μ	Mean distribusi latent, merupakan output dari encoder
σ	Standar deviasi distribusi latent, merupakan output encoder.
ϵ	Noise acak dari $\mathcal{N}(0, I)$ yang disampling tiap forward pass
z	Latent vector hasil sampling
$q_{\varphi}(z x)$	Distribusi posterior latent yang diprediksi encoder
$p(z)$	Distribusi prior, biasanya $\mathcal{N}(0, I)$
D_{KL}	KL Divergence yang mengukur jarak distribusi q dari prior p
L_{rekon}	Mengukur seberapa baik decoder merekonstruksi x dari z

Nama : Putri Torsia Auro Prameswary

NIM : 2546000083

Kelas : Pembelajaran Mesin - B

Halaman ke - 4 .

L_KL Regularisasi agar latent space mendekati distribusi prior

Tabel Koneksi PyTorch

Rumus

Kode PyTorch

μ dan σ dari encoder

`self.fc_mu(h)` dan `self.fc_var(h)`

$z = \mu + \sigma \cdot \epsilon$

`mu + std * torch.randn_like(std)`

$D_{KL}(q(p))$

`KL_loss = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())`

L total

`recon_loss + KL_loss`

4) GAN

$\min_G \max_D V(D, G) = E[\log D(x)] + E[\log(1 - D(G(z)))]$

Variabel

Arti

G

Generator yang bertugas mengubah z menjadi data palsu

D

Discriminator yang memberi skor antara 0 sampai 1

x

Data asli dari dataset

z

Noise vector acak dari $\mathcal{N}(0, 1)$

$G(z)$

Data palsu hasil generator

$D(x)$

Probabilitas D menganggap x sebagai data asli

$D(G(z))$

Probabilitas D menganggap data palsu sebagai data asli

E

Ekspektasi atau rata rata atas distribusi

Tabel Koneksi PyTorch

Rumus

Kode PyTorch

$G(z)$

`fake = generator(z)`

$D(x)$

`real_pred = discriminator(real)`

Loss D

`nn.BCELoss()(pred, label)`

Loss G

`nn.BCELoss()(discriminator(fake), ones)`

5) Diffusion Model (DDPM)

Forward : $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \epsilon$, dengan $\epsilon \sim \mathcal{N}(0, I)$

Loss : $L = \| \epsilon - \epsilon_\theta(x_t, t) \|^2$

Variabel

Arti

x_0

Gambar asli yang bersih dari dataset

x_t

Gambar x_0 yang sudah dikotori noise sampai timestep t

ϵ

Noise acak yang ditambahkan sebagai ground truth

ϵ_θ

Noise yang diprediksi model pada timestep t

t

Timestep yaitu langkah penambahan noise dari 1 sampai T

$\bar{\alpha}_t$

Produk kumulatif noise schedule yang mengatur besar noise di step t

L

MSE antara ϵ asli dan ϵ yang diprediksi model

Tabel Koneksi PyTorch

Rumus

Kode PyTorch

x_t dari proses forward

`sqrt_ab * x0 + sqrt_1mab * eps`

$\epsilon_\theta(x_t, t)$

`unet_model(xt, t)`

Nama : Putri Tosta Aura Prameswary

NIM : 2546000083

Kelas : Pembelajaran Mesin - B

Halaman ke - 5

$L = 1e - \epsilon - 01^2$ `nn.MSELoss()(eps_pred, eps_true)`

6) Text Generation (Autoregressive)

$$p(x_1, x_2, \dots, x_T) = \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1})$$

Variabel Arti

x_t Token pada posisi ke- t dalam urutan

x_1 sampai x_{t-1} semua token sebelum posisi t yang menjadi konteks

$p(x_t | \text{konteks})$ Probabilitas token ke t diberikan konteks sebelumnya

$\prod$ Perkalian probabilitas untuk semua posisi berdasarkan chain rule

T Panjang total urutan token

Tabel Fungsi PyTorch

Rumus

Kode PyTorch

Input konteks x_1 sampai x_{t-1} `input_ids = tokens[:, :-1]`

Target x_t `target_ids = tokens[:, 1:]`

$p(x_t | \text{konteks})$ `logits = model(input_ids)`

Loss `nn.CrossEntropyLoss()(logits, target_ids)`

7) Self Attention (Transformer)

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Variabel Arti

Q Query yaitu representasi token yang bertanya ke token lain

K Key yaitu representasi token yang merespon pertanyaan

V Value yaitu informasi aktual yang diambil berdasarkan skor

d_k Dimensi vektor key, digunakan sebagai faktor normalisasi

QK^T Dot product antara Q dan K yang menghasilkan skor kemiripan antar token

$\sqrt{d_k}$ Pembagi antar nilai skor agar tidak terlalu besar sehingga gradient tidak vanish

softmax Mengubah skor mentah menjadi bobot perhatian dengan jumlah total 1

Tabel Fungsi PyTorch

Rumus

Kode PyTorch

$QK^T / \sqrt{d_k}$ `torch.matmul(Q, K.transpose(-2, -1)) / (d_k ** 0.5)`

$\text{softmax}(\dots)$ `F.softmax(scores, dim = -1)`

dikali V `torch.matmul(weights, V)`

seluruh mekanisme attention `nn.MultiheadAttention()`

Nama : Putri Tarsila Arita Prameswary

NIM : 2546000083

Kelas : Pembelajaran Mesin - B.

Halaman ke-6.

8) Cross Entropy Loss

$$L = -\sum_{i=1}^N y_i \cdot \log(\hat{y}_i)$$

Variabel	Arti
y_i	Probabilitas token target sebenarnya, bernilai 1 untuk token benar dan 0 untuk lainnya
$\hat{y}_i$	Probabilitas token hasil prediksi model yaitu output softmax
N	Jumlah kelas vocabulary
$\log$	Logaritma natural
tanda -	Karena log dari nilai ≤ 1 selalu negatif, dinegasikan agar loss bernilai positif.

Tabel koneksi PyTorch

Rumus	Kode PyTorch
$-\sum y_i \log(\hat{y}_i)$	<code>nn.CrossEntropyLoss()</code>
Softmax sudah termasuk didalamnya	Tidak perlu softmax manual sebelum memanggil <code>nn.CrossEntropyLoss()</code>